Multi-Agent Q-Learning for Real-Time Load Balancing User Association and Handover in Mobile Networks

Alireza Alizadeh *Member, IEEE*, Byungju Lim *Member, IEEE*, and Mai Vu, *Senior Member, IEEE*

arXiv:2412.19835v1 [eess.SP] 22 Dec 2024

***Abstract*—As next generation cellular networks become denser, associating users with the optimal base stations at each time while ensuring no base station is overloaded becomes critical for achieving stable and high network performance. We propose multi-agent online Q-learning (QL) algorithms for performing real-time load balancing user association and handover in dense cellular networks. The load balancing constraints at all base stations couple the actions of user agents, and we propose two multi-agent action selection policies, one centralized and one distributed, to satisfy load balancing at every learning step. In the centralized policy, the actions of UEs are determined by a central load balancer (CLB) running an algorithm based on swapping the worst connection to maximize the total learning reward. In the distributed policy, each UE takes an action based on its local information by participating in a distributed matching game with the BSs to maximize the local reward. We then integrate these action selection policies into an online QL algorithm that adapts in real-time to network dynamics including channel variations and user mobility, using a reward function that considers a handover cost to reduce handover frequency. The proposed multi-agent QL algorithm features low-complexity and fast convergence, outperforming 3GPP max-SINR association. Both policies adapt well to network dynamics at various UE speed profiles from walking, running, to biking and suburban driving, illustrating their robustness and real-time adaptability.**

***Index Terms*—Q-learning, user association, load balancing, handover, user mobility.**

I. Introduction

The ever-increasing demand for higher data rates and lower latencies in beyond 5th generation (B5G) and the upcoming 6th generation (6G) systems pushes for the deployment of dense millimeter wave (mmWave)-enabled cellular networks. These networks have a multi-tier structure composed of multiple base stations (BSs) equipped with different antenna arrays, transmitting at different power levels, operating at different frequency bands from sub-6 GHz to mmWave, and providing different amount of bandwidths. In such dense and heterogeneous networks, network control and resource allocation problems become more complicated and require more efficient approaches.

These dense networks with a high density of BSs intrinsically require user association: the process of assigning each user equipment (UE) to a BS to maximize a network utility. A standing challenge for user association is to satisfy load balancing, that is to keep the number of UEs connected to each BS below its quota (the number of UEs each BS can serve simultaneously), assuming each UE can only connect to one BS at any time instant. If load balancing fails, some BSs may suffer from heavy load condition when many users are connected to that BS. In such cases, user connections could be dropped, or the quality of service of associated users can be significantly degraded due to overloaded traffic at that BS and consequent bottleneck on the backhaul link. Avoiding this scenario necessitates load balancing user association.

This problem, however, is known to be NP-hard due to the presence of integer association variables [1]. The complexity further increases when user association needs to be performed in a highly dynamic network, leading to frequent handovers as a serious concern. As such, there is a need for effective handover algorithms that can adapt to network dynamics including channel variations and user mobility, while maintaining load balancing and achieving high network throughput at a low handover rate.

A. Related and Prior Works

Various analytical approaches have been proposed to address load balancing user association by relaxing the unique association constrains then solving the relaxed problem to obtain association results [1]–[3]. In an earlier work, we introduced a new interference-dependent user association formulation by considering the dependency between user association and interference in mmWave networks [4], which achieves near-optimal solution and has been used as a benchmark in other works including [5], [6]. These algorithms are centralized, hence often exhibiting a high computational complexity and requiring a significant signaling overhead to collect information at a central server. In addition, these works studied the user association problem under a static setting without considering user mobility. In a mobile network, however, the effects of abrupt mmWave channel variations and user mobility on user association and handover can be significant.

This work was supported in part by the National Science Foundation under the CNS/SWIFT Grant 2127648.

A. Alizadeh and M. Vu are with the Department of Electrical and Computer Engineering, Tufts University, Medford, 02155, USA (E-mail: <alireza.alizadeh, mai.vu>@tufts.edu)

B. Lim is with the Department of Electronic Engineering, Pukyong National University, Busan, 48513, South Korea (E-mail: limbj@pknu.ac.kr)

In recent years, machine learning has attracted attention of researchers in wireless communications. Reinforcement learning is a method which can take into account real-time interactions with the system without requiring extensive training, making it particularly suitable for a dynamic wireless system [8]. QL is a specific type of reinforcement learning, which has been applied for user association in the literature, albeit without specific load balancing for all BSs. For example, in dense and static networks, a multi-agent QL model performs for joint power optimization and user association in [9]. Deep QL algorithms are used for user association in LTE network [10], in IoT network [11], and in dense mmWave networks [12]. A centralized handover algorithm using QL achieves link-beam performance gain in [13]. Handover management can use double deep Q-networks for vehicle-to-network communications under high mobility vehicles [14] and for 5G mmwave network [15]. These works did not specify a load constraint for every individual BS, often SBSs can offload to the MBS which does not have a specified load limit.

Recently, several works have considered load balancing using reinforcement learning. For example, a decentralized user association algorithm utilizes a multi-agent actor-critic network in a dense mmWave network, considering handover to reduce energy consumption and delay [16]. A deep reinforcement learning framework is designed to learn handover parameters and antenna tilt angle for distributing cell load evenly where the load is defined as the ratio of allocated resource blocks (RBs) versus the total number of RBs [17]. These works, however, cannot strictly guarantee the load balancing constraints, since these constraints are only considered implicitly and may be violated during online implementation. Other recent works have used multi-armed bandit (MAB) based centralized and semi-distributed algorithms [18] or distributed deep Q-network (DQN) models [19] to strictly satisfy the load balancing constraint for practical implementation. However, MAB based algorithms in [18] output an action without any information about the network state, hence they may not scale well nor provide the best adaptive action under channel variations and user mobility. The DQN models in [19], while adapting to network states, do not consider mobility nor a handover cost, which can result in high handover rates and significant overhead in a highly dynamic network.

B. Our Contributions

We propose online multi-agent QL algorithms for user association and handover in dense and highly-dynamic mmWave networks. Our proposed algorithms explicitly account for the quota at each BS to ensure that the resulting association always satisfy load balancing for every BS at every learning step. The load balancing constraints at BSs introduce dependency among the agents as their actions are coupled, and this dependency is explicitly handled in our proposed algorithms in both centralized and distributed versions. To the best of our knowledge, this is the first work proposing multi-agent QL models for user association and handover that ensure load balancing at all BSs across all tiers, while effectively adapting in real-time to user mobility and channel variations.

Specifically, we formulate the load balancing user association and handover problem as a multi-agent QL system with UEs as agents, by defining the states, actions, and update rule of the Q-values. We use the upper confidence bound (UCB) as the metric for action selection. While our system implements the tabular QL method at each agent, the proposed multi-agent policies and online QL framework also work for deep QL methods. Our main contributions can be summarized as follows.

- We propose two multi-agent action selection policies, a centralized and a distributed version. Both policies ensure the satisfaction of load balancing at all BSs at every learning step. The centralized action selection policy is performed at a central load balancer (CLB) to maximize the total learning rewards. The distributed policy is implemented via a matching game to obtain load-balanced actions. Both policies require no information exchange among agents and have low signaling overheads.
- We propose an online multi-agent QL algorithm integrating the above action selection policies. The proposed online QL algorithm utilizes a reward function that considers a handover cost while maximizing the network sum rate. The signaling overhead and computational complexity of this proposed QL algorithm are analyzed in detail for both the centralized and distributed versions, showing significantly lower overhead for the distributed implementation.
- We perform extensive numerical evaluations in both static and dynamic network settings to assess the convergence rate, performance, and adaptability of the proposed QL algorithms. Results show significant improvements in both throughput and handover rate over the 3GPP max-SINR association method, reaching close to the near-optimal WCS sum rate at a much lower handover frequency. Results also show that the proposed online multi-agent QL algorithms are robust and can adapt well to the dynamics of mmWave channel variation and user mobility under various UE speed profiles including walking, running, biking and suburban driving, validating their use in real-time wireless network.

The rest of this paper is structured as follows. Section II introduces the system model including channel model and UE mobility model, followed by the formulation of an optimization problem for load balancing user association. We present the multi-agent model for Q-learning in Section III. Next, we propose

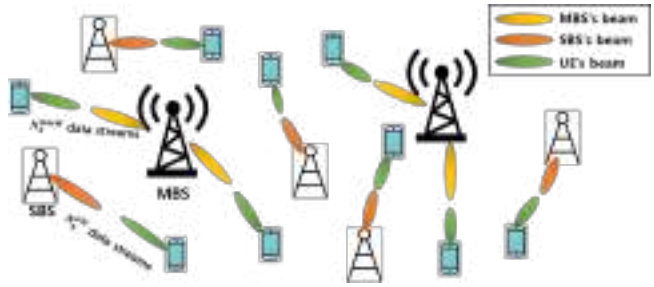


Fig. 1. Illustration of a two-tier cellular HetNet. Each user is associated with one of J_M MBSs and J_s SBSs while requesting N_k^{mmW} data streams from a MBS or $N_k^{\mu\text{W}}$ data streams from a SBS.

the centralized and distributed multi-agent policies for load balancing in Section IV and Section V. Section VI details our online multi-agent QL algorithm for handover utilizing the proposed policies, followed by an analysis of signaling overhead and complexity in Section VII. We evaluate performance of the proposed multi-agent QL algorithm via simulation in Section VIII and provide our conclusions in Section IX.

II. System Models

In this section, we introduce the system models, including the network, channel, and signal models, and define the association variables. Furthermore, we discuss a UE mobility model and frame measurement structure which will enable multi-agent Q-learning to adapt to mobile UEs. While we consider a multi-tier system in this paper, the subsequent proposed learning algorithms can be applied in either a single-tier or multi-tier network.

A. Network and Channel Models

We consider the downlink of a two-tier cellular network including J_M MBSs in tier-1, J_S SBSs in tier-2, and K UEs as shown in Fig. 1, where the MBSs and SBSs operate at sub-6GHz and mmWave, respectively. Let $\mathcal{J} = \{1, ..., j, ..., J\}$ denote the set of all BSs with $J = J_M + J_S$, and $\mathcal{K} = \{1, ..., k, ..., K\}$ represents the set of UEs.

Each BS j is equipped with an antenna array of size M_j and has a specific *quota* m_j representing the maximum number of downlink data streams it can transmit. Thus, the *quota vector* of BSs can be defined as $\mathbf{m} = [m_1, ..., m_J]$. We assume $1 \leq m_j \leq M_j$, where the upper bound is due to the fact that the number of data streams transmitted by each BS cannot exceed the number of its antennas. Each UE is equipped with two antenna arrays: 1) an array with $N_k^{\mu\text{W}}$ elements for sub-6 GHz band, and 2) an array with N_k^{mmW} elements for mmWave band. As such, each UE k can request up to n_k data streams from its serving BS where $n_k \in \{N_k^{\text{mmW}}, N_k^{\mu\text{W}}\}$ depending on the BS type as shown in Fig. 1.

The Gaussian MIMO channel $\mathbf{H}_{k,j}^{\mu\text{W}} \in \mathbb{C}^{N_k^{\mu\text{W}} \times M_j}$ with zero-mean and unit variance i.i.d. complex Gaussian random variables is used for sub-6 GHz transmissions from MBSs. In the mmWave band, however, the transmissions are highly directional due to beamforming at both transmitter and receiver sides and we employ the clustered mmWave MIMO channel $\mathbf{H}_{k,j}^{\text{mmW}} \in \mathbb{C}^{N_k^{\text{mmW}} \times M_j}$ [20], [21]. We use the probability of LoS and NLoS and the path loss model for LoS and NLoS transmissions as given in [20].

B. User Association and Data Rate

The association vector $\boldsymbol{\eta}^{(t)}$ specifies the connections between UEs and BSs at time step t and is defined as

$$\boldsymbol{\eta}^{(t)} \triangleq [\eta_1^{(t)}, ..., \eta_K^{(t)}]^T, \tag{1}$$

where $\eta_k^{(t)} \in \mathcal{J}$ represents the index of BS associated with user k.

Each BS j can transmit a maximum number of m_j data streams to the associated UEs. We formulate this quota by defining a *load balancing* constraint as

$$\sum_{k \in \mathcal{K}} n_k \mathbb{1}_{k,j}^{(t)} \leq m_j, \tag{2}$$

where $\mathbb{1}_{k,j}^{(t)} = 1$ if $\eta_k^{(t)} = j$, and $\mathbb{1}_{k,j}^{(t)} = 0$ if $\eta_k^{(t)} \neq j$.

Highly directional beamforming transmissions and the fast-varying nature of mmwave channels make interference highly dependent on user association [4]. We consider this dependency and formulate the data rate (reward) of UE k from BS j at time step t as [18]

$$\begin{aligned} &R_{k,j}(\boldsymbol{\eta}^{(t)}) \\ &= \log_2 \left[\det \left(\mathbf{I}_{n_k} + \left(\mathbf{V}_{k,j}(\boldsymbol{\eta}^{(t)}) \right)^{-1} \mathbf{W}_k^* \mathbf{H}_{k,j} \mathbf{F}_{k,j} \mathbf{F}_{k,j}^* \mathbf{H}_{k,j}^* \mathbf{W}_k \right) \right], \end{aligned} \tag{3}$$

where $\mathbf{V}_{k,j}(\boldsymbol{\eta}^{(t)})$ is the interference-plus-noise covariance matrix given as

$$\begin{aligned} \mathbf{V}_{k,j}(\boldsymbol{\eta}^{(t)}) =& \mathbf{W}_k^* \mathbf{H}_{k,j} \left(\sum_{l \in \mathcal{K}_j^{(t)}, l \neq k} \mathbf{F}_{l,j} \mathbf{F}_{l,j}^* \right) \mathbf{H}_{k,j}^* \mathbf{W}_k \\ &+ \mathbf{W}_k^* \left(\sum_{i \in \mathcal{J}, i \neq j} \sum_{l \in \mathcal{K}_i^{(t)}} \mathbf{H}_{k,i} \mathbf{F}_{l,i} \mathbf{F}_{l,i}^* \mathbf{H}_{k,i}^* \right) \mathbf{W}_k + N_0 \mathbf{W}_k^* \mathbf{W}_k. \end{aligned} \tag{4}$$

Here $\mathbf{F}_{k,j} \in \mathbb{C}^{M_j \times n_k}$ is the linear precoder (transmit beamforming matrix) at BS j intended for UE k, and $\mathbf{W}_k \in \mathbb{C}^{N_k \times n_k}$ is the linear combiner (receive beamforming matrix) of UE k. Set $\mathcal{K}_j^{(t)}$ is the activation set of BS j at time t, which includes all UEs associated with the BS at that time. Eq. (4) shows the dependency of interference on user association. We note that there is no inter-tier interference as the two tiers are operating at two separate frequency bands.

The total learning reward at each time step t is computed as the overall *network sum rate* given by

$$r(\boldsymbol{\eta}^{(t)}) = \sum_{j \in \mathcal{J}} \sum_{k \in \mathcal{K}} R_{k,j}(\boldsymbol{\eta}^{(t)}), \tag{5}$$

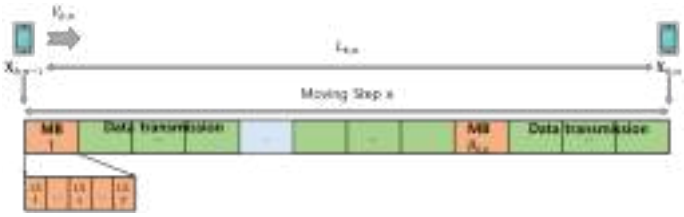


Fig. 2. Structure of moving step n during which UE k travels from source waypoint $\mathbf{X}_{k,n-1}$ to target waypoint $\mathbf{X}_{k,n}$ with velocity $V_{k,n}$. The number of MBs for each moving step is obtained according to (6), which depends on UE velocity $V_{k,n}$, distance between source and target waypoints $L_{k,n}$, and time duration of each MB t^{MB}.

which will also be used as a measure of network performance.

C. User Mobility Model

In a mobile network, UEs are moving and keeping track of the best associations is challenging. In a mmWave-enabled network, this problem is even more challenging since mmWave channel conditions can change abruptly in short periods of time. Next, we consider a mobility model and introduce a measurement model which will be used in Sec. VI to study handover in such a highly-dynamic network.

For UE movements, we utilize the *modified random waypoint* (MRWP) model, which is defined by an infinite sequence of quadruples as $\{(\mathbf{X}_{k,n-1}, \mathbf{X}_{k,n}, V_{k,n}, Z_{k,n})\}_{k\in\mathcal{K},\ n\in\mathbb{N}}$, where n denotes the *moving step* n during which UE k travels from starting waypoint $\mathbf{X}_{k,n-1}$ to target waypoint $\mathbf{X}_{k,n}$ [22]. In this mobility model, each UE k uniformly selects a random velocity $V_{k,n} \in (0, V_{\max}]$, and a random pause time $Z_{k,n}$ at the target waypoint. Then, given a source waypoint $\mathbf{X}_{k,n-1}$, the UE generates a homogeneous Poisson point process (PPP) $\Phi(n)$ with intensity λ, and selects the nearest point in $\Phi(n)$ as the target waypoint, i.e., $\mathbf{X}_{k,n} = \arg\min_{\mathbf{x}\in\Phi(n)} ||\mathbf{x} - \mathbf{X}_{k,n-1}||$. Thus, the *transition length* of UE k during moving step n can be calculated as $L_{k,n} = ||\mathbf{X}_{k,n} - \mathbf{X}_{k,n-1}||$, and its *transition time* is $T_{k,n} = L_{k,n}/V_{k,n}$.

We assume each moving step is composed of multiple *measurement blocks* (MBs), interspersed between data transmission blocks as shown in Fig. 1. Inside each MB, each UE performs a set of measurements from its serving and neighbor BSs. The measurement quantities, as specified in the 3GPP standards, can be the reference signal received power (RSRP), reference signal received quality (RSRQ), or SINR [23]. These measurements are then reported to the network every t^{MB} seconds[1] for the purpose of handover. Thus, the number of MBs for UE k during moving step n is determined by rounding up the ratio between the transition time of the UE and the report interval as follows:

$$B_{k,n} = \left\lceil \frac{T_{k,n}}{t^{\text{MB}}} \right\rceil. \tag{6}$$

Thus, $B_{k,n}$ is random and changes from one moving step to another. At the end of each MB, the network can make handover decisions based on the new measurements, which may lead to changes in associations.

D. Optimization Problem

The goal is to find the optimal association vector $\boldsymbol{\eta}^{(t)}$ at each learning step, which maximizes a network utility function. In this paper, we consider network throughput (5) as the utility function and define the user association problem at learning step t as

$$\underset{\boldsymbol{\eta}^{(t)}}{\text{maximize}} \quad r(\boldsymbol{\eta}^{(t)}) \tag{7a}$$

$$\text{subject to} \quad \sum_{j\in\mathcal{J}} \mathbb{1}_{k,j}^{(t)} \le 1, \quad \forall k \in \mathcal{K} \tag{7b}$$

$$\sum_{k\in\mathcal{K}} n_k \mathbb{1}_{k,j}^{(t)} \le m_j, \quad \forall j \in \mathcal{J} \tag{7c}$$

The set of constraints in (7b) represents the unique association constraints, meaning that each UE can only connect to one BS at a time. The constraints in (7c) allows our association scheme to limit each BS's load separately. The load balancing quota (m_j) is set by the available resources at each BS, and this set of constraints guarantees that each BS can serve all its associated UEs up to the specified load.

Under load balancing constraints, the associations of all UEs are inter-dependent. This problem can be solved by a centralized algorithm such as WCS [4], which make association decisions by considering the dependency among UEs while satisfying load balancing constraints. However, a centralized optimization algorithm requires significant computational complexity and high signaling overhead such as delivering the measured SINR values and global channel state information (CSI) to a central server. Moreover, performing optimization in a highly dynamic network may be impractical, since the optimization needs to be resolved at each measurement block b to effectively adapt to the network dynamics. In this paper, we propose a multi-agent reinforcement learning approach for user association, which can reduce signaling overhead by using only local information at each user and adapting in real-time to changes in the environment.

III. Multi-agent QL Model For User Association

User association can be cast as a multi-agent QL (QL) problem in which UEs (or UEs) are agents, selecting a BS corresponds to taking an action, the wireless networks represents the environment, and maximizing the network utility is the long-term goal. In a multi-agent QL system, ideally the agents take

[1]This parameter is known as information element *ReportInterval* in 3GPP standardization, which indicates the interval between periodical reports. The range of value for the ReportInterval in 5G NR is 120 ms - 30 min [23].

actions independently using only local information. The load balancing constraints in (7c), however, introduce an important dependency among the actions taken by different agents.

There have been MARL (multi-agent reinforcement learning) architectures which deal with the dependency among different agents' actions. These include introducing communications among agents such as CommNet [24] and IC3Net [25], or using Actor-Critic-type algorithms with centralized training and decentralized execution such as MADDPG [26] and MAAC [27]. The former approach of introducing communications among agents would require device-to-device communications among UEs, which we do not assume here. The latter approach of centralized training and decentralized execution allows agents to observe some implicit dependency, but does not guarantee that a constraint such as load balancing can be met at every execution step.

In this paper, we propose a multi-agent QL system which guarantees load balancing at every execution step. The proposed QL system employs a multi-agent action selection policy to ensure load balancing, which is described in the subsequent two sections. Next, we define the parameters of the QL model at each agent, and introduce the Q-value update rule at each agent and the multi-agent actions selection metric based on the upper confidence bound (UCB).

A. Cellular Network QL Components

The components of our proposed QL-based user association model are defined as follows. Each UE represents an *agent* which interacts with the cellular network to achieve a goal. The considered QL model therefore is a multi-agent system with K agents. Each UE k perceives its environment through a set S_k consisting of states of the form $s_k = (\eta_k, \boldsymbol{\rho}_k) \in \mathcal{S}_k$, where $\eta_k \in \mathcal{J}$ is the association variable specifying the BS serving (connecting with) UE k, and $\boldsymbol{\rho}_k$ is a vector of size J denoting the quantized SINR values from UE k to the J considered BSs, given current connection state with BS η_k. The η_k^{th} element of $\boldsymbol{\rho}_k$ represents the quantized SINR level of the BS associated with UE k, and all other elements denote the SINR levels of other BSs not associated with UE k in this state. In practice, SINR values are measured for each data stream of UE k, and vector $\boldsymbol{\rho}_k$ is obtained by averaging the SINR values over all data streams at this UE.

We use two sets of SINR quantization levels: S values ($S > 2$) for the associated BS where the quantization is uniform between an SINR^{min} and an SINR^{max} value, and binary quantization (2 values) for other BSs as these are less sensitive for performance. As a result, the size of state space for each UE is $|\mathcal{S}_k| = 2^{J-1}JS$.

At each learning step t, UE k realizes a specific state $s_k^{(t)} = \left(\eta_k^{(t)}, \rho_k^{(t)}\right) \in \mathcal{S}_k$. The association vector $\boldsymbol{\eta}^{(t)} = \left[\eta_1^{(t)}, ..., \eta_K^{(t)}\right]$ contains the current association states of all UEs. At each step t, UE k takes a specific action $a_k^{(t)} \in \mathcal{J}$ and transitions from state $s_k^{(t)}$ to state $s_k^{(t+1)} = \left(\eta_k^{(t+1)}, \rho_k^{(t+1)}\right)$, where $\eta_k^{(t+1)} = a_k^{(t)}$. After taking an action $a_k^{(t)}$, each UE k receives an instantaneous reward $R_{k,a_k^{(t)}}$ as in (3), which will be used to determine its next SINR state and update its Q-value.

B. Measurement Model and Learning Steps

In this subsection, we define the learning process and association process by utilizing a frame structure that contains multiple measurement blocks as in Fig. 2. Recall from the mobility model that each UE's moving step contains multiple measurement blocks as defined in (6). Each measurement block (MB) b contains a fixed number of learning steps, denoted as T. We assume that channel state information (CSI) is static within an MB, but dynamically changes in the next MB. At the beginning of each MB, each UE (agent) k takes a measurement of its SINR vector $\rho_k^{(b)}$. During learning step t, each agent obtain its current state $s_k^{(t)}$, chooses an action, and update its chosen Q-value (as discussed in the next subsections). Even though the CSI is static during each MB consisting of T learning steps, the current chosen Q-value of each UE will change in each learning step as shown later in the updating rule (9), thereby facilitating the learning process.

The actions from all agents at each learning step t produce a new learning association vector $\boldsymbol{\eta}^{(t)}$, which will be updated through the multi-agent policy described in Sections IV and V to ensure load balancing at every BS. This learning association vector, however, is not used for performing association and handover. Instead, we keep another association vector $\boldsymbol{\beta}^{(b)}$, the *best-to-date association*, which is updated at the end of each measurement block b, after every T learning steps, and which will be used for performing connections (association and handover). We note that while it is also possible to update the best-to-date association and perform connections at the end of each learning step, this may result in less stable associations and more frequent handovers.

We note also that the assumption of fixed CSI per measurement block is not essential for our QL algorithms to work. In a fast fading environment, it is possible to have CSI varied during an MB, in which case each UE will perform multiple measurements during each MB. Specifically, each UE can perform one measurement of its $\rho_k^{(t)}$ per learning step, for a total of T measurements in each MB. The role of T in both cases of slow and fast fading is to allow the update of best-to-date association vector after a number of learning steps, rather than at every learning step to reduce handover frequency.

Specifically, each measurement block b has an association vector $\boldsymbol{\beta}^{(b)}$, chosen from all learning association

vectors $\boldsymbol{\eta}^{(t)}$ within that measurement block as

$$\boldsymbol{\beta}^{(b)} = \arg\max_{t=\{1,\ldots,T\}} r(\boldsymbol{\eta}^{(t)}), \tag{8}$$

where $r(.)$ is given in (5) and the initial learning association vector at each MB b is the best-to-date association vector from the previous MB $b-1$, that is $\boldsymbol{\eta}^{(1)} = \boldsymbol{\beta}^{(b-1)}$. In this way, $\boldsymbol{\beta}^{(b)}$ is the best-to-date association vector taking into account all past history.

By using these two different association vectors, we distinguish between the learning process and the association process. This is done by performing two separate updating procedures: 1) behavioral update (or learning update of $\eta_k^{(t)}$), and 2) target update (or operation/association update of $\beta_k^{(b)}$). The behavioral procedure updates $\eta_k^{(t)}$ at each learning step (as discussed later in (9)), adapting to the immediate changes in the environment, whereas the target procedure updates $\beta_k^{(b)}$ only when the new association vector results in a higher network sum as in (8), thus aiming at increasing the long-term expected reward. In a sense, our update of the best-to-date association vectors is similar to the target Q-network update in [7], except that $\beta_k^{(b)}$ is updated as best-to-date instead of periodically, and is used for performing the actual association connection instead of learning.

C. Q-value Update at Each UE

To focus our attention on the multi-agent policies design (Sections IV and V), we choose standard tabular-based Q-value update in this paper. The proposed multi-agent action selection policies, however, are also applicable to deep Q-networks (DQN), which can handle a large state-action space. In other words, our Q-learning framework can be applied with any training and updating method, including either standard-QL [28] or deep Q-learning [7], [29].

In standard QL, each UE k locally maintains and updates a *Q value table* $\mathbf{Q}_k$ containing the Q-values for all its state-action pairs. The table has the size of $|\mathcal{S}_k| \times J$ (total number of states $\times$ number of actions). Each entry of this Q-table is called a Q-value $Q_k(s_k, a_k)$ which is the current estimate of the long-term expected reward for taking action a_k in state s_k. Since the size of the state-action space increases exponentially with the number of BSs, for large networks, each Q-table can be replaced by a deep Q-network (DQN). In DQN, a DNN (deep neural network) is used at each UE to compute the Q-values for all possible actions given an input state. Since the input size only grows linearly with the number of BSs, DQN can handle a large state-action space. We will not discuss DQN implementation in this paper, but note that our proposed multi-agent policies (Sections IV and V) and online algorithm (Section VI) can be applied to DQNs as well.

At each learning step t, when user k in current state $s_k^{(t)}$ takes an action $a_k^{(t)}$, the value for this state-action pair is updated in this UE's Q-table as follows [8]:

$$Q_k\left(s_k^{(t+1)}, a_k^{(t+1)}\right) = (1-\alpha) Q_k\left(s_k^{(t)}, a_k^{(t)}\right) + \alpha\left[\tilde{R}_{k,a_k} + \gamma \max_{b_k \in \mathcal{J}} Q_k\left(s_k^{(t+1)}, b_k\right)\right], \tag{9}$$

where $\alpha \in [0,1)$ is the learning rate, $\gamma \in [0,1)$ is the discount factor for future rewards, b_k represents all possible actions in the next state, among which the best is chosen, and $\tilde{R}_{k,a_k}$ is the immediate reward for agent k by taking action a_k in the current state. The choice of the reward function is important and will be discussed in Section VI.

D. Multi-agent Actions Selection Metric

In order to determine which entry of the Q-table of each UE to be updated as in (9), we need a *policy* which defines how an UE k chooses the action $a_k^{(t)}$ given its current state. The policy needs a metric to select the actions.

Different metrics have been used in the literature for deciding the action at an agent. Here we choose the UCB (upper confidence bound), instead of the direct Q value, as the metric for action selection. The UCB metric allows a balance between *exploiting* the current best action and *exploring* other actions [8]. For each UE k at current state $s_k^{(t)}$, the UCB value (or a U-value) for a state-action pair is defined as follows

$$U_k\left(s_k^{(t)}, a\right) = Q_k\left(s_k^{(t)}, a\right) + c\sqrt{\frac{\ln t}{N_k^{(t-1)}\left(s_k^{(t)}, a\right)}}, \tag{10}$$

where $c > 0$ controls the degree of exploration, and $N_k^{(t-1)}\left(s_k^{(t)}, a\right)$ is the number of times action a has been selected at state $s_k^{(t)}$ up to and including learning step $t-1$. Each time an action a is taken by UE k, this number is updated as follows

$$N_k^{(t)}\left(s_k^{(t)}, a\right) = N_k^{(t-1)}\left(s_k^{(t)}, a\right) + 1. \tag{11}$$

In the next sections, we will discuss how the U-values in (10) are updated and used in the multi-agent actions selection policy to ensure load balancing at every step.

IV. Centralized Multi-agent Actions Selection Policy for Load Balancing

In the centralized approach, a central entity determines the actions for all agents to ensure load balancing at every BS. We propose the use of a central entity called the *CLB* to manage actions selection. The CLB runs an algorithm to maximize the total UCB values for all UEs while satisfying the load balancing constraints in (2). This CLB produces a load-balanced association vector $\boldsymbol{\eta}^{(t)}$ at each learning step and sends the corresponding association result to each UE.

A. CLB Record Keeping and Updates

In order to perform centralized load balancing, the CLB needs to collect the Q-values from all UEs for all state-action pairs. This collection needs to occur *only once* at the initialization of the learning process, during which each UE sends its Q-table to the CLB. After this initialization, at each learning step, each UE reports a single Q-value to the CLB for the specific state-action pair updated at the UE in that step as in (9). The CLB then updates and maintains its record of the Q-tables from all UEs. Note also that each UE also maintains and updates its own Q-table. In other words, the CLB keeps a record of K Q-tables of all K UEs, and updates one entry, $Q_k\left(s_k^{(t)}, a_k^{(t)}\right)$, in each table at each learning step, when that entry is also updated in the Q-table at the corresponding UE. We emphasize that the Q-values are not shared among agents, but are only shared with the CLB, facilitating a multi-agent QL without information exchange among agents.

Similarly, the CLB maintains a set of N-tables of all K UEs, which is initialized to 0 at the beginning of the learning process. Note that the N-tables are only kept at the CLB but not at the UEs, and there is no communication between UEs and the CLB on the N-values. At each learning step, the CLB updates one entry of each N-table according to the state-action pair of the corresponding UE as in (11).

At each learning step, the CLB extracts a row from each of its Q-tables and N-tables record to compute a network U-table corresponding to the UEs' current state $\mathbf{s}^{(t)} = \left[s_1^{(t)}, \ldots s_k^{(t)}, \ldots, s_K^{(t)}\right]$. Specifically, the CLB forms the network U-table of size $K \times J$ consisting of one row for each UE as follows:

$$\mathbf{U}(\mathbf{s}^{(t)}) = \left[\mathbf{u}_1^T\left(s_1^{(t)}\right); \ldots \mathbf{u}_k^T\left(s_k^{(t)}\right); \ldots \mathbf{u}_K^T\left(s_K^{(t)}\right)\right], \qquad (12)$$

where the U-values are computed using the Q-values and N-values as in (10). The CLB then uses this network U-table to perform load balancing assignment, using *the CLB load balancing algorithm* proposed below, to obtain a new learning association vector. Then, the CLB informs all the UEs about their load-balanced association, which determines their action to be taken.

When a UE carries out this action, it receives a reward, updates its Q-value, and reports this value to the CLB. Then, the CLB updates the corresponding entry in the Q-table and N-table of that UE.

B. CLB Load Balancing Action Selection Policy

As discussed above, in each learning step t, the CLB receives an updated Q-value from each UE. The CLB uses each reported value to update its Q-table for the corresponding UE, and also calculate the update in its N-table for that UE. The UEs need not send any N-value. Next, the CLB produces the U-table as in (12), which contains the U-values of the current learning step for all possible connections. Then, the

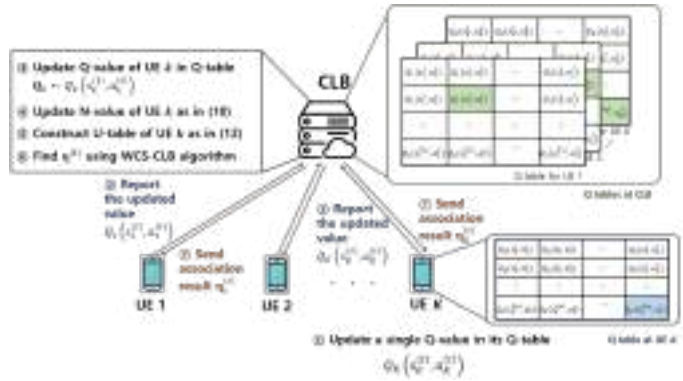


Fig. 3. Procedure for Q-value update at the CLB in the proposed centralized action selection policy

CLB aims to find the association vector which achieves the maximum sum U-value for this learning step. This problem is an integer programming, whose optimal solution would require an exhaustive search which is exponential in complexity. We use the idea of WCS in [4] which is a polynomial-time method to achieve a high sum U-value at the CLB while satisfying load balancing. The principal idea of WCS is to start with an association vector that is load-balanced, then swap the worst connection with another connection to improve the U-value sum while maintaining load balancing. The worst connection under an association vector $\boldsymbol{\eta}^{(i)}$ is the one with lowest U-value in $\mathbf{U}^{(t)}$. For each swapping step i, we find a new association vector $\boldsymbol{\eta}^{(i+1)}$ to maximize sum U-value as follows.

$$\boldsymbol{\eta}^{(i+1)} = \underset{\{\boldsymbol{\eta}_n^{(i)} | n \in \mathcal{K},\ n \neq k^{(i)}\},\ \boldsymbol{\eta}^{(i)}}{\arg\max} \sum_{k \in \mathcal{K}} \mathbf{U}^{(t)}_{k, \eta_k^{(i)}}. \qquad (13)$$

where $\boldsymbol{\eta}_n^{(i)}$ is the new association vector by swapping the worst connection UE $k^{(i)}$ with the connection of UE n. In addition, we apply a *switching step* to explore other possibilities for improving the objective function. If $\boldsymbol{\eta}^{(i+1)} = \boldsymbol{\eta}^{(i)}$, we compute a UE switching index $l^{(m)} = \mathrm{mod}(m-1, K) + 1$ and switch the $k^{(i+1)^{\text{th}}}$ and $l^{(m)^{\text{th}}}$ elements of the current association vector to obtain the switched association vector $\boldsymbol{\eta}_{\text{sw}}^{(i+1)}$. This new association vector is used as the initial association vector for the next swapping iteration. The process repeats until no improvement in the U-value sum is found. The proposed algorithm guarantees convergence because the objective in (13) is always increasing. The detailed WCS-CLB algorithm is summarized in Alg. 1. Fig. 3 illustrates the complete procedure for Q-value updates and association vector selection at the CLB.

Remark 1: The CLB-WCS algorithm starts with an initial association vector which is chosen to satisfy load balancing at all BSs. Then during the process, each swapping or switching step does not change the load at any BS, but simply swaps or switches the UEs associating with two different BSs. Thus the resulting association vector at every step of the CLB-WCS algo-

Algorithm 1: (WCS-CLB) Centralized Action Selection Policy

Input: Q values $Q_k(s_k^{(t)}, a_k^{(t)}), \forall k \in \mathcal{K}$ reported from all UEs, and quota vector $\mathbf{m}$

Initialization:

- Initialize $i = 1$ and $m = 1$;
- The CLB forms the network U-table $\tilde{\mathbf{Q}}^{(t)}\left(\mathbf{s}^{(t)}\right) = \left[\tilde{\mathbf{q}}_1^{(t)}\left(s_1^{(t)}\right), ..., \tilde{\mathbf{q}}_K^{(t)}\left(s_K^{(t)}\right)\right]$;
- Generate an arbitrary feasible association vector $\boldsymbol{\eta}^{(1)}$ according to $\mathbf{m}$;
- Find the initial worst connection $\left(k^{(1)}, j^{(1)}\right) = \arg\min_{k,j} \tilde{\mathbf{Q}}^{(t)}_{k,\eta_k^{(1)}}$;

while *stopping criterion is not met* **do**

  Swap the element $k^{(i)}$ with all other elements of $\boldsymbol{\eta}^{(i)}$ to obtain new vectors $\boldsymbol{\eta}_n^{(i)}$;

  Find the best association vector $\boldsymbol{\eta}^{(i+1)} = \arg\ \max_{\{\boldsymbol{\eta}_n^{(i)} | n \in \mathcal{K},\ n \neq k^{(i)}\},\ \boldsymbol{\eta}^{(i)}} \sum_{k \in \mathcal{K}} \tilde{\mathbf{Q}}^{(t)}_{k,\eta_k^{(i)}}$;

  Find the worst connection $\left(k^{(i+1)}, j^{(i+1)}\right) = \arg\min_{k,j} \tilde{\mathbf{Q}}^{(t)}_{k,\eta_k^{(i+1)}}$;

  if $\boldsymbol{\eta}^{(i+1)} = \boldsymbol{\eta}^{(i)}$ **then**

    $l^{(m)} \leftarrow \mathrm{mod}(m-1, K) + 1$;

    Switch the elements $k^{(i+1)}$ and $l^{(m)}$ of $\boldsymbol{\eta}^{(i+1)}$ to obtain $\boldsymbol{\eta}_{\mathrm{sw}}^{(i+1)}$;

    $\boldsymbol{\eta}^{(i+1)} \leftarrow \boldsymbol{\eta}_{\mathrm{sw}}^{(i+1)}$;

    $m \leftarrow m + 1$;

  end

  Stopping criterion: Break if the best $\boldsymbol{\eta}$ does not change after K consecutive iterations;

  $i \leftarrow i + 1$;

end

Set the learning association vector $\boldsymbol{\eta}^{(t)} = \boldsymbol{\eta}^{(i)}$;

CLB informs each UE k about its $\eta_k^{(t)}$;

Output: Learning association vector $\boldsymbol{\eta}^{(t)}$

Algorithm 2: (MG-DLB) Distributed Action Selection Policy

Input: Q values $Q_k(s_k^{(t)}, a_k^{(t)}), \forall k \in \mathcal{K}$ reported from all UEs, and quota vector $\mathbf{m}$

Initialization:

- Set $n = 1$ and $l_k = 1,\ \forall k$;
- Each UE k builds its preference lists ($\mathcal{P}_k^{\mathrm{UE}}$) using its U-vector $\tilde{\mathbf{q}}_k^{(t)}$;
- Each BS j builds its preference lists ($\mathcal{P}_j^{\mathrm{BS}}$) using its U-values $\tilde{Q}_k\left(s_k^{(t)}, j\right)$;
- Form the initial rejection set $\mathcal{R} = \{1, 2, ..., K\}$;
- Initialize the waiting list of each BS $\mathcal{W}_j^0 = \varnothing,\ \forall j$;
- Perform a DA matching game [30] as follows:

while $\mathcal{R} \neq \varnothing$ **do**

  Each UE $k \in \mathcal{R}$ applies to its l_k^{th} preferred BS in $\mathcal{P}_k^{\mathrm{UE}}$;

  Each BS j forms its current waiting list $\mathcal{W}_j^n$ from its new applicants and its previous waiting list $\mathcal{W}_j^{n-1}$;

  Each BS j keeps the first m_j preferred UEs according to $\mathcal{P}_j^{\mathrm{BS}}$, and reject the rest of them;

  for $k \in \mathcal{R}$ **do**

    $l_k \leftarrow l_k + 1$;

    if $l_k > J$ **then**

      Remove UE k from $\mathcal{R}$;

    end

  end

  $n \leftarrow n + 1$;

end

Each BS informs all the UEs in its waiting list about the final association decision;

Output: Distributed learning association results $\eta_k^{(t)}$ at UE $k,\ k \in \mathcal{K}$

rithm always satisfies load balancing at all BSs. That is, by starting with a load-balanced initial association, CLB-WCS guarantees an output association vector satisfying all load balancing constraints.

Remark 2: During the execution of the WCS-CLB algorithm at each learning step, the network U-table $\mathbf{U}\left(\mathbf{s}^{(t)}\right)$ remains fixed at the CLB and no communications between UEs and the CLB occur. Communications between UEs and the CLB occur only before and after each execution of the WCS-CLB. Before each run of the WCS-CLB, each UE sends a single Q-value to the CLB for updating and computing the U-table in (12). After each WCS-CLB execution, the CLB delivers the association results $\{\eta_k^{(t)}\}$ by sending a single associated BS index to each UE. As such, the communication requirement between UEs and the CLB is minimal.

V. Distributed Multi-Agent Actions Selection Policy for Load Balancing

For the distributed approach, we propose a distributed game based on matching theory in which UEs apply to individual BSs and receive a response for association action which ensures load balancing of the responding BS. In this distributed approach, no central entity exists and there are no messages or information exchanges among agents; the user association algorithm operates in a fully distributed manner.

A. Distributed Record and Update of Q- and U-tables

Here there is no central entity which maintains the Q-tables of all UEs. Instead, each UE and each BS stores and maintains its own copy of a single Q-table in a distributed fashion. Specifically, each UE maintains and updates its Q-table and N-table locally. At each learning step t, UE k builds its own U-vector $\mathbf{u}_k^{(t)}\left(s_k^{(t)}\right)$ based on (10) and (11), and sorts the values in this vector in descending order to form its current preference list.

Each BS j also maintains and updates (based on UEs' reports) its local record of a single U-table $\mathbf{U}_j$ composed of K columns, one from each UE corresponding to the action of connecting to this BS j. Assuming the state space of all agents is of the same size $|\mathcal{S}_1| = ... = |\mathcal{S}_k| = ... = |\mathcal{S}_K|$, the size of $\mathbf{U}_j$ is $|\mathcal{S}_k| \times K$. This assumption is only for simpler notation but is not necessary for the algorithm to work; each BS can maintain K U-vectors, each of a specific size $|\mathcal{S}_k|$.

At each learning step, each BS j receives up to m_j reports of a new U-value, each from one of its associated UEs, and uses these reports to update its table $\mathbf{U}_j$. Then the BS selects the current U-values $\left[U_k\left(s_k^{(t)}, j\right),\ \forall k \in \mathcal{K}\right]$ and sorts them in descending or-

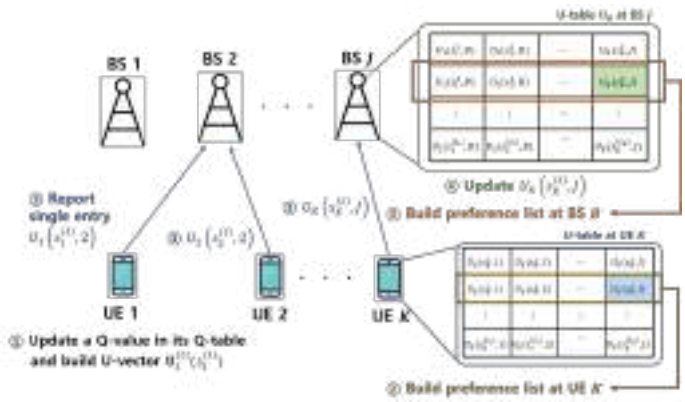


Fig. 4. Procedure for U-value update in the proposed distributed action selection policy without CLB.

der to form its preference for that time step. The U-value update procedure is illustrated in Fig. 4.

We emphasize that the U-tables maintained at a UE and at a BS are not the same: the U-table at BS j is made up of K different columns, each equals to the jth column of the U-table at UE k, $(k = 1, ..., K)$. Different from the centralized approach, here no entity has all Q-tables or N-tables of all the UEs.

B. MG-DLB Load Balancing Action Selection Policy

We propose a matching game distributed load balancing (MG-DLB) algorithm. At each learning step t, the UEs and BSs play a distributed deferred acceptance (DA) matching game [30] which leads to load-balanced association resulting from each BS controls how many UEs to admit based on its load balancing constraint. From the updated U-tables at each UE and each BS, the UEs and the BSs build their own preference list using the U-value vector corresponding to the current state of each UE. Each UE will have a preference list of length J, and each BS a preference list of length K. All UEs and BSs then participate in the matching game in a distributed fashion, using only the locally stored U-table without the need for a central entity. Note that the U-table at each BS and each UE is fixed during a game, but will change in the next learning step.

The distributed game works as follows. At each learning step, each UE has a preference list based on its sorted U-vector for its current state $s_k^{(t)}$. Each UE applies first to the BS at the top of its preference list by sending a request to this BS. Each BS also has a preference list based on its sorted U-vector for all UEs' current states $\{s_k^{(t)}\}$. Each BS will wait-list the top D_j requests based on its own preference list, where D_j is its load balancing limit, and reject the rest. Each rejected UE then applies to the next BS on its preference list. The process continues until all UEs have either been wait-listed at a BS or been rejected by all BSs. At that point, each BS commits to admiting all UEs in its waitlist, which provides the association results. Each game can include up to J application-response rounds between the UEs and BSs. Since the waitlist at BS j only includes up to D_j requests, load balancing at each BS is always guaranteed. A summary of this distributed load balancing is shown in Alg. 2.

Remark 3: The MG-DLB is run in a distributed fashion at each UE and each BS and requires no physical host or control entity. This is in contrast to the WCS-CLB algorithm which is run centrally at the CLB. During an execution of the MG-DLB algorithm, the UEs perform an application-response process with the BSs, where there are at most J application-response rounds. At the end of an MG-DLB execution, each UE receives an association if it is in the final waiting list of a BS, otherwise the UE is unassociated. Each UE then takes the resulting action, and reports the updated U-value for the taken state-action pair to its associated BS. All the updating and reporting of U-values for MG-DLB are done in a fully distributed fashion.

VI. Online Multi-agent QL Algorithm for Seamless Handover

In this section, we integrate the above multi-agent decision policies into an *online multi-agent QL algorithm* that performs user association and handover seamlessly, considering user mobility and handover cost.

In a dynamic network, user mobility and channel variations can cause UEs to handover from a serving BS to a target BS. The handover process interrupts data transmission and hence incurs a cost, which is usually translated to a reduction in the data rate. To capture the effect of handover in our proposed algorithm, we apply a handover cost in the learning process.

We now propose an online QL algorithm which can be implemented in either centralized or distributed fashion by selecting the corresponding load balancing action selection policy as described in Sec. IV and Sec. V. The proposed algorithm provides the best-to-date association $\boldsymbol{\beta}$ at any moving block while the learning process continues indefinitely in the background.

A. Initialization

At the beginning of the algorithm, each UE randomly initializes its Q-table. For the centralized implementation, each UE sends its entire Q-table to CLB, and the CLB initializes K N-tables as $\mathbf{N}_k = \mathbf{0}, \forall k \in \mathcal{K}$. Since the CLB performs UCB updates and action selections, UEs do not need to keep their N-tables. For the distributed approach, however, each UE k also initializes its N-tables as $\mathbf{N}_k = \mathbf{0}$, then computes its U-table and sends each column of its U-table to the corresponding BS.

B. Online QL Execution and Updates

The algorithm starts by selecting a load balancing action selection policy. Centralized or distributed learning can be carried out by employing WCS-CLB (Alg.

Algorithm 3: Online QL for User Association and Handover

Input: Learning rates α, discount factor γ, BSs' quota vector $\mathbf{m}$, randomly generated initial Q-values $Q_k\left(s_k^{(0)}, j\right)$, $\forall k \in \mathcal{K}, s_k \in \mathcal{S}_k, \forall j \in \mathcal{J}$, initialize N-tables $\mathbf{N}_k^{(0)} = \mathbf{0}, \forall k \in \mathcal{K}$

Initialization:

Reporting initial Q-values:;
- If centralized: each UE sends its initial Q-table to CLB;
- If distributed: each UE sends each column of its initial Q-table to the corresponding BS;

Repeat for each moving step n:
for *each measurement block (MB)* b **do**
  for $t = 1 : T$ **do**
    Perform load balancing assignment using WCS-CLB (centralized Agl. 1) or MG-DLB (distributed Alg. 2);
    Access and Mobility Management Function (AMF):
    Collects UEs' data rates to calculate network throughput $r(\boldsymbol{\eta}^{(t)})$ according to (5);
    if $r(\boldsymbol{\eta}^{(t)}) > r(\boldsymbol{\beta})$ **then**
      Updates the best-to-date association vector $\boldsymbol{\beta}^{(b)} = \boldsymbol{\eta}^{(t)}$;
      Performs necessary handovers based on $\boldsymbol{\beta}^{(b)}$ for data transmission;
    end
    Each UE k:
    Takes an action $a = \eta_k^{(t)}$ which is the next association state;
    Receives reward $R_{k,a}^{(t)}$ and specifies its SINR state from serving BS;
    Updates its Q-value for UA (using (9)) or for UA-HO (using (14));
    Reports $Q_k(s_k^{(t)}, a)$ to the CLB (centralized) or to each BS a (distributed);
  end
  Output: Best-to-date association vector $\boldsymbol{\beta}^{(b)}$
end

1) or MG-DLB (Alg. 2), respectively. The online QL algorithm is run continuously after initialization. Each MB b consists of T learning steps, and during each learning step, each UE k performs the following tasks.

First, it takes an action based on its learning association ($a = \eta_k^{(t)}$). In the centralized implementation, UEs are informed about their learning association by the CLB, but in the distributed implementation each UE knows its learning association as the result of the matching game. After taking the action, the UE receives a reward from the selected BS and specifies its SINR state.

Next, the UE updates its Q-value. To incorporate a handover cost, we set the reward in (9) by considering a connection-time dependent handover cost. In particular, a handover cost is incurred for a UE if its new learning association differs from the previous learning association (i.e., $\eta_k^{(t)} \neq \eta_k^{(t-1)}$), resulting in a HO-reward $\tilde{R}_{k,a_k^{(t)}}$ obtained as

$$\tilde{R}_{k,a_k^{(t)}} = \left(1 - \zeta(\tau_k^{(b)})\bar{\delta}(\eta_k^{(t-1)}, a_k^{(t)})\right) R_{k,a_k^{(t)}}. \tag{14}$$

Here, $a_k^{(t)} = \eta_k^{(t)}$, and $\zeta(\tau_k^{(b)}) = C_d e^{-\frac{\tau_k^{(b)}}{10}} + C_0$ represents the handover cost where C_d is a soft-cost, C_0 is a hard-cost, and $\tau_k^{(b)}$ is the *sojourn time*, the time duration during which the UE has remained associated with its current serving BS up to t [18], and $R_{k,a_k^{(t)}}$ is the data rate obtained from the connection as in (3). The HO-reward in (14) is used in (9) for updating Q-values.

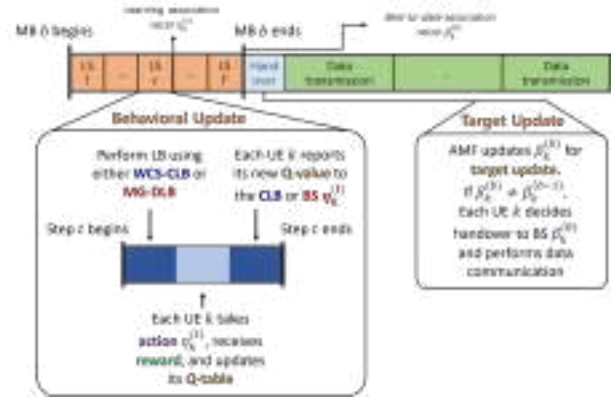


Fig. 5. Timing diagram for one measurement block (MB) b, showing the behavioral (learning) update and the target (association) update. Each learning step t within MB b runs one inner iteration of the load balancing policy and the updating and reporting process, as described in Alg. 3. After T learning steps, each UE performs handover if $\beta_k^{(b)} \neq \beta_k^{(b-1)}$, followed by data communication between each UE and its associated BS.

Finally, the UE reports its updated Q-value to the CLB in the centralized implementation, or computes its updated U-value and report this update to the associated BS in the distributed implementation (see Figs. 3 and 4). Here, a Q-value (or U-value) is updated by each UE at every learning step, while each moving step of a UE is composed of multiple measurement blocks, each block with T learning steps. As such, the proposed algorithm has the ability to adapt to each UE's mobility, which will be clearly verified in the simulation section. This integrated online QL algorithm is summarized in Alg. 3.

C. Performing Handovers

During the online QL process, 5G core access and mobility management function (AMF)[2] maintains the best-to-date association vector $\boldsymbol{\beta}^{(b)}$. At each learning step t, the new learning association vector $\boldsymbol{\eta}^{(t)}$ resulting from the WCS-CLB or MB-DLB algorithm is reported to the AMF. The AMF also collects the user data rates (or SINR values) and calculates the network throughput under the resulting learning association vector. If there is an improvement compared to the throughput under the best-to-date association, then the AMF updates the best-to-date association vector as $\boldsymbol{\beta}^{(b)} = \boldsymbol{\eta}^{(t)}$. At the end of MB b, each UE k performs handover to BS $\beta_k^{(b)}$, and carry out data communication

[2]The AMF is a central entity in 5G core which receives all connection related information from UEs and is responsible for handover and mobility management tasks [31]. The functionality of AMF in 5G is similar to mobility management entity (MME) in LTE evolved packet core (EPC).

with this new BS. Note that the vector $\boldsymbol{\beta}^{(b)}$ refers to the best association vector only within MB b, not of all time. Fig. 5 summarizes the process of action decision making and association updating in online multi-agent QL in each MB.

We note that the AMF is not engaged in the learning process, and is only responsible for collecting all current data rates from the UEs to decide about updating the best-to-date associations for data transmission. We thus separate this built-in AMF function from the learning process which can be either centralized (using WCS-CLB) or fully distributed (using MG-DLB).

VII. Signaling Overhead and Complexity Analysis

A. Signaling Overhead

The signaling overhead of the proposed QL algorithm depends on its centralized or distributed implementation. For the centralized version of Alg. 3, after performing the load balancing algorithm (WCS-CLB) in each learning step, the CLB informs each UE about its association for learning $\eta_k^{(t)}$ and the AMF informs each UE about its association for data transmission (only if $\beta_k^{(b)}$ changes). Assuming each signaling of an association variable requires X_2 bits, the total number of bits for reporting association results is KX_2. Then, each UE reports its updated Q-value for the taken (state, action) pair to the CLB. Assuming this reporting requires X_1 bits each, the total number of bits sent from all UEs to the CLB is KX_1. Thus, the maximum (since $\boldsymbol{\beta}$ is not necessarily signaled in every step) signaling overhead per learning step for the centralized load balancer is $X^{\text{CLB}} = K(X_1 + 2X_2)$ bits. In addition, for updating the best-to-date association, the AFM collects the user data rates or SINR values to compute the sum rate, which requires $X^{\text{AMF}} = KX_3$ bits, assuming that the data rate information is quantized with X_3 bits.

For the distributed implementation using MG-DLB, since the preference lists of UEs and BSs remain unchanged during game, there is no extra signaling other than the one-bit application and response messages which require the maximum (worst case) signaling of $2KJ$ bits. If any $\beta_k^{(b)}$ changes, the AMF informs UEs about their associations for data transmission, resulting in KX_2 bits. For reporting, all UEs must report their Q-value to the corresponding BS (instead of reporting them to the CLB). Thus, the maximum signaling for this distributed implementation is $X^{\text{DLB}} = K(X_1 + X_2 + 2J)$ bits.

In comparison, the centralized, non-learning WCS algorithm in [4] requires global network information including instantaneous CSI, which must be collected to a central server. Assuming each element of the channel matrix is quantized with X_4 bits, the signaling overhead per measurement for the WCS algorithm is $X^{\text{WCS}} = KJM_jN_kX_4$. In this non-learning WCS algorithm, each UE must report M_jN_k complex-valued numbers in each MB to each BS. In contrast, the proposed learning algorithm only requires each UE to report a single real U-value (or Q-value) to only one BS (or the CLB) in each learning step, leading to a total of T real-valued numbers reported in each MB. Typically, T is small (around 10 or fewer per MB, as shown later in Sec. VIII.B.1), whereas the product of the numbers of BSs and antennas JM_jN_k can be huge for mmWave communication. Thus, the complexity of the non-learning WCS algorithm is significantly higher than either the centralized or distributed QL implementation. The overall comparisons of signaling overhead and complexity are summarized in Table. I.

TABLE I
Comparison of Signaling Overhead and Complexity

Scheme	Signaling overhead	Computational Complexity
WCS [4]	$KJM_jN_kX_3$	$O(M_j^2K^2\log(K))$
QL-WCS-CLB	$K(X_1+2X_2)$	$O(K^2\log K)$
QL-MG-DLB	$K(X_1+X_2+2J)$	$JO(KJ\log K)+KO(J\log J)$

B. Computational Complexity

Since the Q-value updates in Alg. 3 (Lines 14-16) are simple scalar multiplication and addition operations, the computation complexity of the proposed Q-leaning algorithm is dominated by its load balancing algorithm. In what follows, we calculate the cost of each load balancing algorithm separately.

For WCS-CLB (Alg. 1), the most computationally consuming steps are Lines 7. Line 7 involves two operations: computing $K + 1$ sum-rewards which is in the order of $O(K)$, and sorting the resulted sum-rewards which costs $O(K\log K)$, giving the total cost of $O(K\log K)$ at each *while* loop. Since the number of iterations of the while loop linearly increases with K [4], the overall cost of Alg. 1 is of $O(K^2\log K)$.

Executing the DA game in MG-DLB (Alg. 2) incurs a small sorting cost at each player of the game. In Line 2, each UE sorts its U-vector to build its preference list at a cost of $O(J\log J)$. At each iteration, each BS sorts its applicants to update its waiting list, a procedure which incurs a maximum cost of $O(K\log K)$ since the maximum number of applicants for each BS is K. The number of iterations in a DA game is at most J [32], resulting in the overall complexity of $O(KJ\log K)$ at each BS. Since UEs do not perform sorting, the overall complexity at each UE is $O(J\log J)$. This gives Alg. 2 a total complexity of $JO(KJ\log K) + KO(J\log J)$, albeit distributed among all BSs and UEs. Since usually $J \ll K$, the computation complexity for MG-DLB is much lower than for WCS-CLB.

In short, the signaling overhead of MG-DLB is comparable with that of WCS-CLB, while its computation complexity is much lower than the centralized load balancer and is distributed among all BSs and UEs.

TABLE II
NETWORK SETTING FOR SIMULATION RESULTS

Network	J_M	J_S	K	Quota vector of BSs
1	1	3	18	$\mathbf{m} = [18, 6, 6, 6]$
2	2	4	30	$\mathbf{m} = [18, 18, 6, 6, 6, 6]$
3	2	4	60	$\mathbf{m} = [36, 36, 12, 12, 12, 12]$

VIII. NUMERICAL RESULTS

In this section, we evaluate the performance of our proposed online QL user association and handover algorithm in the downlink of a two-tier HetNet. The MBSs and SBSs operate at 1.8 GHz and 28 GHz, respectively. The channels for sub-6 GHz links and mmWave links (composed of 5 clusters and 10 rays per cluster) are generated per MB as described in Sec. II-A. To enable 3D beamforming at BSs, each BS is equipped with a UPA of size 8×8 ($M_j = 64$). The number of sub-6 GHz and mmWave antennas at each UE k is $N_k^{\mu W} = 2$ and $N_k^{\text{mmW}} = 4$, respectively. We also assume that the transmit power of MBS is 10dBm higher than that of SBSs. The noise power spectral density is -174 dBm/Hz, and the respective available bandwidths at each MBS and SBS are 20 MHz and 400 MHz, respectively. We consider three network settings for simulations as summarized in Table II. `Network 2` is the base network used in all simulations. Network nodes are deployed in a 500×500 m^2 square where the BSs are placed at specific locations and the UEs are distributed randomly according to a homogeneous Poisson point process (PPP). For learning algorithms implementation, fixed values for learning rate $\alpha = 0.9$ and discount factor $\gamma = 0.2$ are picked based on extensive evaluations.

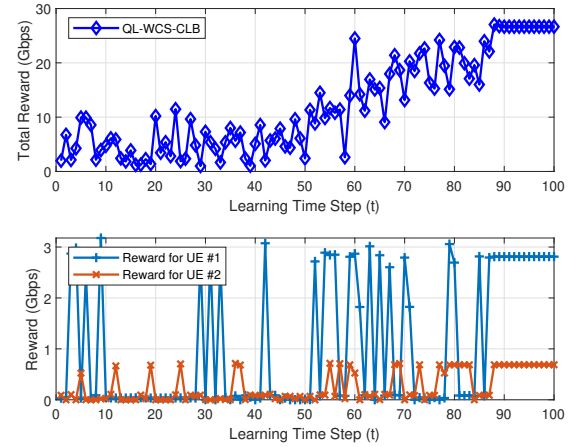
Throughout this section, the proposed QL algorithms are compared with the following benchmarks:

- **WCS** [4]: Centralized optimization (non-learning) algorithm for load-balancing association achieving near optimal network sum rate. WCS must collect global network CSI and be re-run for each measurement update per MB (see Table I), hence is used as an upper bound for network sum rate performance.
- **3GPP Max-SINR** [31]: Each UE coonects to the BS providing the highest max-SINR, and we drop users if a BS becomes overloaded to ensure load balancing.

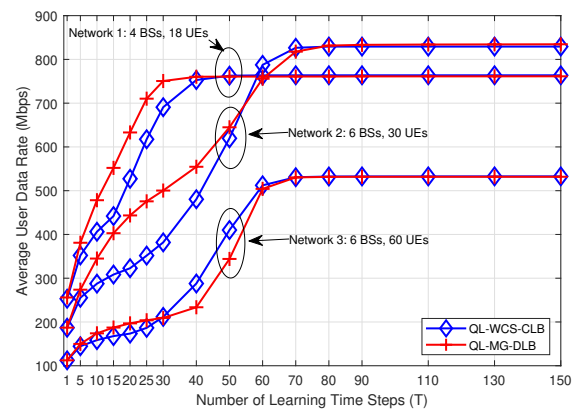
A. Learning Performance in Static Networks

First we consider static networks in which users are stationary with random but fixed locations (no mobility). We implemented both the centralized and distributed versions of the proposed QL algorithm: i) `QL-WCS-CLB` which employs the WCS-CLB algorithm for load balancing (Alg. 1), and ii) `QL-MG-DLB` which performs the MG-DLB algorithm for distributed load balancing (Alg. 2).

1) Learning convergence and achievable data rate: Fig. 6.a shows convergence plots of the total expected reward of all UEs (upper figure) and the expected rewards of two typical UEs (lower figure) with respect to learning steps for `QL-WCS-CLB` in `Network 2`. It is clear that the total reward improves as learning step grows. The results show that the algorithm converges in a reasonable number of learning steps (around 86 in this setting).



(a) Total expected rewards of all UES (upper) and expected reward of two typical UEs (lower) for the proposed centralized QL algorithm (`QL-WCS-CLB`)



(b) The average user data rate of the proposed centralized (`QL-WCS-CLB`) and distributed (`QL-MG-DLB`) QL algorithms

Fig. 6. Convergence behavior of the proposed algorithms in static networks (no UE mobility).

Fig. 6.b depicts the best-to-date average per-user data rate versus the number of learning steps for the centralized and distributed QL algorithms for the networks defined in Table II. Interestingly, for all network sizes, the distributed `QL-MG-DLB` performs better than the centralized `QL-WCS-CLB` for a low number of learning steps. The convergence for both the centralized and distributed QL algorithms is similar and quite fast; for example, in `Network 1`, both algorithms reach the maximum performance after only 40 steps. Also, it is worth noting that the average per-user data rate in `Network 3` is lower than `Network 2`, while both the number of UEs and quota of BSs are doubled. This is due to the fact that, the increased number of

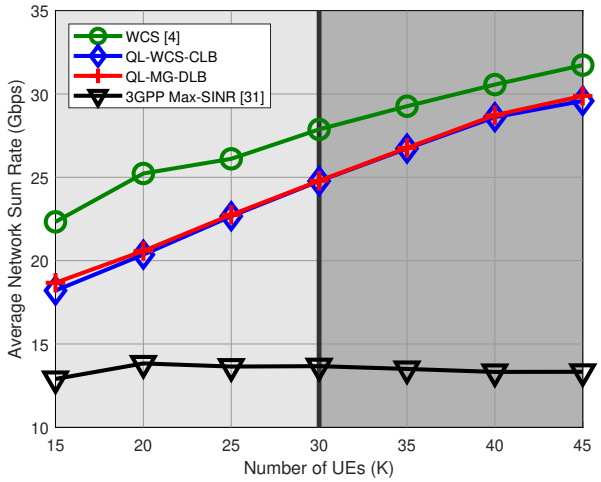


Fig. 7. Effect of different loading scenarios on the average network throughput in a static `Network 2` with $T = 100$ (no UE mobility). The transmit powers at MBSs and SBSs are 45 dBm and 35 dBm.

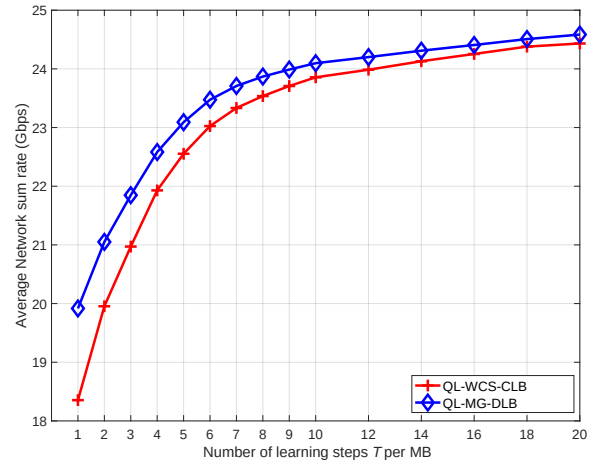


Fig. 8. Effect of the number of learning steps T per MB, on the network throughput in dynamic `Network 2` with UE mobility. In each moving step, 30% of UEs are moving, each with a random velocity $V_{k,n} \in \{1, ..., 10\}$ m/s. The data rate values are captured after 5 moving steps, when the throughput starts to converge to a stable value.

UEs in `Network 3` resulted in higher interference in the network, and hence lower per-user data rate. The sum rate among all users or the network throughput, however, is the highest for `Network 3`.

These results show that our multi-agent QL algorithms are well converged even though the agents do not share any information, nor communicate, nor have access to common information. The convergence of multi-agent QL when only local information is exploited at each agent has, in generally, not been proven theoretically, but only shown via empirical means [33]. In the same manner, here we show the convergence of our proposed algorithms via simulation. Due to the joint action decision at the CLB in `QL-WCS-CLB` or at the BSs in `QL-MG-DLB`, convergence of the proposed algorithms is guaranteed even though no common state information is available to the agents.

2) Learning adaption to network load: Fig. 7 evaluates throughput performance of the proposed QL algorithms under different network loading scenarios for `Network 2`: 1) underload ($K < 30$), 2) critical load ($K = 30$), and 3) overload ($K > 30$).

Both the centralized and distributed QL algorithms show similar performance, and outperform `3GPP Max-SINR` algorithm by between 48-130%. They also reach closely the performance of `WCS` algorithm, with a gap of about 9% under light load scenario (15 UEs) and 4% under heavy load scenario (45 UEs). For all learning-based algorithms, we observe that the network throughput increases with more users, even under overload scenarios, while `3GPP Max-SINR` exhibits an almost flat network throughput which is slightly decreasing when overloading. This result illustrates the robustness and effectiveness of the proposed learning algorithms to associate users even under overload scenarios to keep the throughput increase while satisfying load balancing.

B. Handover in Dynamic Networks

Next we evaluate the performance of the proposed online QL algorithm, Alg. 3, in dynamic networks where UEs are mobile and can experience handover. For these simulations, we focus on `Network 2`, and assume that the duration of each MB is $t^{\text{MB}} = 480$ ms, and in each moving step, 30% of UEs are moving.

1) Number of learning steps per block: Fig. 8 evaluates the effect of number of learning steps T per MB on the average network throughput after $n = 5$ moving steps, when the network throughput starts to converge to a stable value. It can be observed that the distributed QL algorithm slightly performs better especially at low values of T. The results show that at small values of T, increasing the number of learning steps T per block has a significant impact on increasing the network throughput, as more steps provides the learning algorithms with more data for real-time updates. This increase in throughput, however, diminishes as T increases beyond around 10 steps per block. For practical purposes, T should be kept relatively small to reduce system overhead. Results in Fig. 8 suggests that $T = 6$ is a reasonable value to achieve good throughput performance, thus we use $T = 6$ in each MB in all subsequent simulations.

2) Mobility trajectories and handovers example: Under mobility at walking speed ($V = 6$ km/h) with random initial UE locations, Fig. 9 shows the UE trajectories and associations of the UEs along their movement trajectories. The figure shows multiple instances where specific users switch their associations along their trajectory, hence handovers occur. It can also be observed that the associated BS of a UE is not always the closest BS. Take UE number 13 for example, during its movements, the UE switches associations from the orange BS (initial BS) to red then to red and

then to pink, in order to satisfy load balancing. Note that since the proposed online QL algorithms takes into account the handover cost as in (14), the average handover frequency is in fact kept to a minimum, as will be verified in Fig. 11.

3) Real-time network throughput at different mobility speeds: Fig. 8 compares the network throughput of the proposed online multi-agent QL algorithm in a static network and dynamic networks with different UE mobility categories: 1) walking (V = 6 km/h), 2) biking (V = 17 km/h), and 3) suburban driving (V = 40 km/h). All dynamic networks start from an initial state of complete rest with random initial load-balanced associations. The figure shows network throughputs achieved by online learning Alg. 3 in real time, averaged over different sets of initial locations of UEs.

The proposed algorithms achieve a network throughput in dynamic settings close to that in the static setting (at 94% throughput for V = 6 km/h). The figure also shows that faster user mobility speed requires more moving steps for convergence. The initial ramping up during the first few moving steps is due to a limited number of measurements and short history at the beginning, and as the number of MBs increases, network throughput performance improves because the learning process has accumulated a longer and richer history. A higher mobility speed incurs a more dynamic Q-table (or U-table), but the proposed online QL algorithm can adapt to these dynamics by updating Q-values (or U-values) at every learning step, converging to a stable performance with accumulated learning history. While there is a decrease in network throughput with the increase in user mobility speed as expected, the online QL algorithm is able to converge and reach a stable throughput at each mobility speed after at most 4 moving steps.

4) Throughput and handover frequency comparison: Fig. 11 depicts the average network throughput and average handover rate of several user association schemes versus moving steps. For this simulation, we implemented the distributed algorithm `QL-MG-DLB` with the reward function in (14), while for the centralized version `QL-WCS-CLB` we replaced $\eta_k^{(t-1)}$ in (14) with $\beta_k^{(b-1)}$ as in [18] as it provides the lowest handover rate based on extensive simulations.

The proposed QL algorithms significantly outperform the `3GPP Max-SINR` scheme in both network throughput and handover rate. Fig. 11.a shows that the centralized and distributed QL algorithms achieve a network throughput within 87% and 89% respectively of the benchmark `WCS`, while almost doubling the throughput achieved by 3GPP. Fig. 11.b shows that the proposed learning algorithms achieve significantly lower handover rates than both `WCS` and `3GPP Max-SINR`, at around an order of magnitude lower than that of 3GPP.

Thus compared to 3GPP, the proposed QL algorithm reduces frequent handover by a factor of 10 while almost doubling the achievable throughput in mmWave

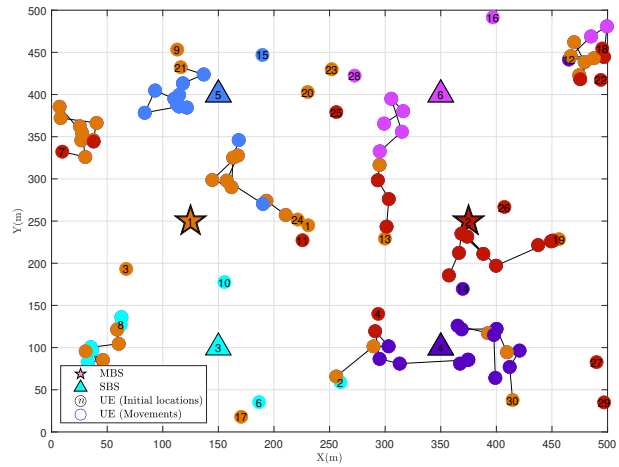


Fig. 9. A layout example of dynamic `Network 2` where each UE is labeled with a number at its initial location. In this setting, 9 UEs (30% of UEs) move randomly at each moving step at V = 6 km/h (walking). Each small circle represents the location of a UE at a particular way point, and the color of the circle corresponds to the color of the associated BS at that way point. A change of color along a UE's trajectory signifies a handover.

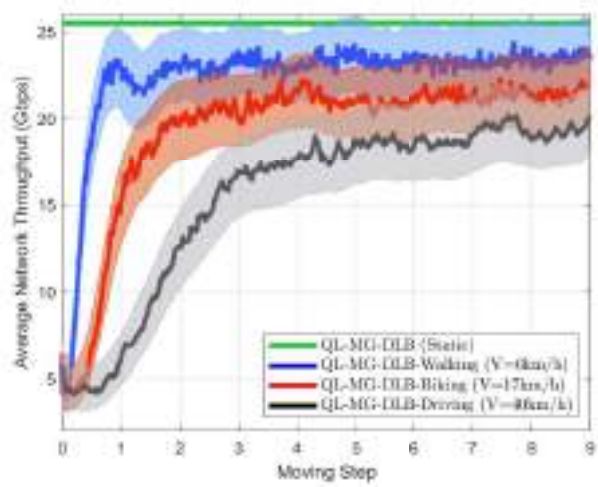


Fig. 10. Effect of mobility speed on the convergence of network throughput in dynamic `Network 2`, where shaded areas indicate standard deviations. For each network at a particular speed setting, 30% of UEs are moving at any one time.

networks. These results point to a high level of real-time adaptability and effectiveness of the proposed QL algorithms with handover under user mobility. Furthermore, the proposed QL algorithms achieve better handover frequency than WCS at a lower network sum rate. This presents an interesting trade-off between network sum rate and handover frequency.

IX. Conclusion

We proposed online multi-agent QL algorithms for user association and handover in dense mmWave networks, where each user acts as an agent employing either a centralized or a distributed load balancing policy to ensure all BSs load balancing. In the centralized

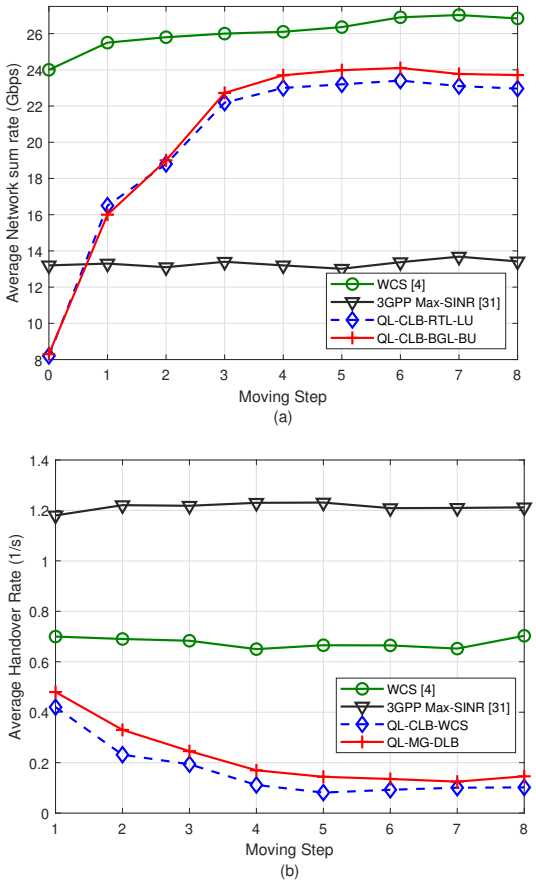


Fig. 11. Comparisons of a) the average network throughput rate, and b) the average handover rate of the proposed online QL Alg. 3 in constrast with `WCS` [4] and `3GPP Max-SINR` algorithms in a dynamic `Network 2` where 30% of UEs are moving at each moving step with a random velocity $V_{k,n} \in \{1, ..., 10\}$ m/s.

approach, the action selection for UEs is done by a CLB, while in the distributed approach, each UE takes an action based on its local information via playing a distributed matching game. We apply a mobility and measurement model and integrate a handover cost in the reward for users. Numerical results show significant network throughput and handover rate improvements over the conventional 3GPP max-SINR scheme, where the throughput reaches close to that of the benchmark WCS algorithm at a significantly lower handover rate. Both the proposed centralized and distributed algorithms featured low-complexity and fast convergence, making them suitable for real-time operation in highly dynamic networks.

References

[1] Q. Ye et al., "User association for load balancing in heterogeneous cellular networks," *IEEE Trans. Wireless Commun.*, vol. 12, no. 6, pp. 2706–2716, 2013.

[2] D. Bethanabhotla, O. Y. Bursalioglu, H. C. Papadopoulos, and G. Caire, "Optimal user-cell association for massive MIMO wireless networks," *IEEE Trans. Wireless Commun.*, vol. 15, no. 3, pp. 1835–1850, 2016.

[3] E. M. Taghavi, R. Hashemi, A. Alizadeh, N. Rajatheva, M. Vu, and M. Latva-aho, "Joint active-passive beamforming and user association in irs-assisted mmwave cellular networks," *IEEE Trans. Veh. Technol.*, pp. 1–14, 2023.

[4] A. Alizadeh and M. Vu, "Load balancing user association in millimeter wave mimo networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 6, pp. 2932–2945, 2019.

[5] P. Han, Z. Zhou, and Z. Wang, "User association for load balance in heterogeneous networks with limited csi feedback," *IEEE Commun. Lett.*, vol. 24, no. 5, pp. 1095–1099, 2020.

[6] S. Khosravi, H. S. Ghadikolaei, and M. Petrova, "Learning-based load balancing handover in mobile millimeter wave networks," in *Proc. IEEE Globecom*, Dec 2020, pp. 1–7.

[7] V. Mnih et al., "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. The MIT Press, 2015.

[9] D. Li et al., "User Association and Power Allocation Based on Q-Learning in Ultra Dense Heterogeneous Networks," in *Proc. IEEE GLOBECOM*, Dec. 2019.

[10] N. Zhao et al., "Deep reinforcement learning for user association and resource allocation in heterogeneous cellular networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 11, pp. 5141–5152, 2019.

[11] Q. Zhang, Y.-C. Liang, and H. V. Poor, "Intelligent user association for symbiotic radio networks using deep reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 19, no. 7, pp. 4535–4548, Jul 2020.

[12] M. Sana, A. De Domenico, W. Yu, Y. Lostanlen, and E. C. Strinati, "Multi-agent reinforcement learning for adaptive user association in dynamic mmwave networks," *IEEE Trans. Wireless Commun.*, vol. 19, no. 10, pp. 6520–6534, Oct 2020.

[13] V. Yajnanarayana, H. Rydén, and L. Hévizi, "5G handover using reinforcement learning," in *Proc. IEEE 3rd 5G World Forum (5GWF)*, 2020, pp. 349–354.

[14] K. Tan, D. Bremner, J. Le Kernec, Y. Sambo, L. Zhang, and M. A. Imran, "Intelligent handover algorithm for vehicle-to-network communications with double-deep Q-learning," *IEEE Trans. Veh. Technol.*, vol. 71, no. 7, pp. 7848–7862, July 2022.

[15] C. Lee, J. Jung, and J.-M. Chung, "Intelligent dual active protocol stack handover based on double DQN deep reinforcement learning for 5G mmwave networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 7, pp. 7572–7584, Jul 2022.

[16] J. Moon, S. Kim, H. Ju, and B. Shim, "Energy-efficient user association in mmwave/thz ultra-dense network via multi-agent deep reinforcement learning," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 692–706, Jun 2023.

[17] H.-H. Chang, H. Chen, J. Zhang, and L. Liu, "Decentralized deep reinforcement learning meets mobility load balancing," *IEEE/ACM Trans. Networking*, vol. 31, no. 2, pp. 473–484, Apr 2023.

[18] A. Alizadeh and M. Vu, "Reinforcement learning for user association and handover in mmwave-enabled networks," *IEEE Trans. Wireless Communications*, vol. 21, no. 11, pp. 9712–9728, Nov 2022.

[19] B. Lim and M. Vu, "Distributed multi-agent deep Q-learning for load balancing user association in dense networks," *IEEE Wireless Commun. Lett.*, vol. 12, no. 7, pp. 1120–1124, Jul 2023.

[20] 3rd Generation Partnership Project (3GPP), "Study on channel model for frequencies from 0.5 to 100 GHz," Technical Report 38.901, Jun. 2018, v. 15.0.0.

[21] T. A. Thomas, H. C. Nguyen, G. R. MacCartney, and T. S. Rappaport, "3D mmWave channel model proposal," in *Proc. IEEE 80th Veh. Technol. Conf. (VTC Fall)*, 2014, pp. 1–6.

[22] X. Lin, R. Krishna Ganti, P. J. Fleming, and J. G. Andrews, "Towards understanding the fundamentals of mobility in cellular networks," *IEEE Trans. Wireless Commun.*, vol. 12, no. 4, pp. 1686–1698, 2013.

[23] 3rd Generation Partnership Project (3GPP), "5G NR Radio Resource Control (RRC); Protocol specification," Technical Specification 38.331, May. 2022, v. 17.0.0.

[24] S. Sukhbaatar, A. Szlam, and R. Fergus, "Learning multiagent communication with backpropagation," *NIPS*, vol. 29, 2016.

[25] A. Singh, T. Jain, and S. Sukhbaatar, "Learning when to communicate at scale in multiagent cooperative and competitive tasks," *arXiv preprint arXiv:1812.09755*, 2018.

[26] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," *Advances in neural information processing systems*, vol. 30, 2017.

[27] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *International conference on machine learning*. PMLR, 2019, pp. 2961–2970.
[28] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
[29] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, no. 1, 2016.
[30] D. Gale and L. S. Shapley, "College admissions and the stability of marriage," *The American Mathematical Monthly*, vol. 69, no. 1, pp. 9–15, 1962.
[31] 3GPP, "System Architecture for the 5G System," 3rd Generation Partnership Project (3GPP), Technical Specification 23.501, Jun. 2020, version 15.12.0.
[32] A. Alizadeh and M. Vu, "Distributed user association in B5G networks using early acceptance matching games," *IEEE Trans. Wireless Commun.*, vol. 20, no. 4, pp. 2428–2441, Apr. 2021.
[33] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *Proceedings of the tenth international conference on machine learning*, 1993, pp. 330–337.